BIOT 6900 · AI-POWERED BIOTECH WORKFLOWS

Module 2 Lab Guide — Multi-Omics Target Identification & Validation

Week 2 · Computational Lab 2 · Fall 2026

How today works

Today runs as a short lecture followed by three hands-on parts. They're scaffolded on purpose — you watch the method first, then we run it together on real data, then you do it yourself:

Stage	What happens	Mode
Lecture	The concepts — why we integrate omics layers, and the five ideas below.	listen
Part 1	I walk through the worked example on synthetic breast-cancer data.	watch
Part 2	Together, we find and load real CPTAC data, then re-run Part 1 on real numbers.	guided
Part 3	You complete the graded assignment on Alzheimer's data.	you do

> **Only Part 3 is graded.**
> Parts 1 and 2 are there to get the method into your hands on data where the answer is already understood. Part 3 is the assignment you submit.

What this lab is

You'll integrate three data layers — transcriptomics (RNA), proteomics (protein), and genomics — to nominate and **rank disease targets**. This is the keystone of Part I of the course: the target list you build in Part 3 is what Week 3 (druggability) and Week 4 (structure prediction) both build on.

> **The one idea to carry through**
> How you integrate is dictated by what your data has matched. Matched → correlate within samples. Unmatched → compare gene-level summaries. Same goal, different machinery.

Maps to course outcomes: LO1 (apply industry tools to biological data), LO2 (integrate multimodal data), LO3 (interpret results in biological context), LO5 (communicate through professional documentation).

The lecture, in five beats

Keep these ideas in mind through all three parts:

- **Why integrate at all.** Each layer can produce artifacts the others don't share, so a target supported across genome, transcriptome, and proteome is a stronger bet than one seen in only one layer.
- **Concordance and discordance both carry signal.** A gene up at RNA but flat at protein isn't noise — it's post-transcriptional biology. Learn to read disagreement, not just reward agreement.
- **The identifier problem.** Gene symbols, Ensembl IDs, and UniProt accessions don't line up for free. Harmonizing datasets to one key is where most of the real work lives.

- **Integration granularity is forced by the data.** Sample-matched → correlate within patients (Parts 1–2). Unmatched → compare summary statistics per gene (Part 3).
- **Prioritization is a scoring decision.** Turning three noisy layers into one ranked list means defining a multi-evidence score. Every weight you choose is a judgment call you must justify.

Before you start

1 · Your environment & packages

Reuse the `biot6900` environment from Module 1. Module 2 uses the same core scientific stack plus Jupyter — activate the environment and install (or confirm) these packages:

```
conda activate biot6900
conda install -y numpy pandas scipy jupyter ipykernel nbconvert
```

If you set your environment up with pip / a virtual environment instead of conda, run this from your project folder:

```
python -m venv .venv            # first time only
source .venv/bin/activate       # Windows: .venv\Scripts\activate
pip install numpy pandas scipy jupyter ipykernel nbconvert
```

Then launch the notebook, and confirm the core packages import with no error:

```
jupyter notebook              # or: jupyter lab

# this should print three version numbers, no errors:
python -c "import numpy, pandas, scipy; print(numpy.__version__, pandas.__version__,
scipy.__version__)"
```

Stuck on setup? Don't lose an evening — bring your laptop to office hours **before** the deadline. Most install problems take a few minutes to fix in person.

2 · The data, by part

Part 1 (provided). Download the three synthetic CPTAC files from Canvas (`cptac_brca_rna.tsv`, `cptac_brca_protein.tsv`, `cptac_brca_mutation.tsv`) into a `data/` folder next to the notebook. These are simulated teaching data — engineered so the method is easy to see. (If they're missing, the notebook auto-generates its own labelled demo, so Part 1 always runs.)

Part 2 (we get together). We'll download real CPTAC breast data from LinkedOmics in class — recipe in the Part 2 section.

Part 3 (Alzheimer's — required). Download the three Alzheimer's matrices posted on Canvas into the same `data/` folder. The notebook expects these exact names and columns:

File	Columns
`ad_transcriptomics.tsv`	`gene` `log2fc` `pval`
`ad_proteomics.tsv`	`gene` `log2fc` `pval`
`ad_gwas.tsv`	`gene` `neglog10p`

All three join on the `gene` column. Create the folder once with `mkdir data` (or make it in your file browser) and drop the files in — your folder should look like:

```
your-repo/
├── BIOT6900_Module2_Starter.ipynb
└── data/
    ├── cptac_brca_rna.tsv        (Part 1)
    ├── cptac_brca_protein.tsv    (Part 1)
    ├── cptac_brca_mutation.tsv   (Part 1)
    ├── ad_transcriptomics.tsv    (Part 3)
    ├── ad_proteomics.tsv         (Part 3)
    └── ad_gwas.tsv               (Part 3)
```

Part 1 — Worked example on synthetic data (watch)

I'll run this in front of you. **Nothing here is graded** — the goal is to see the whole method work on clean, matched data before you touch real numbers. The breast tumors are measured at all three layers, so samples line up and we integrate **at the sample level.** Follow along in the notebook.

Step 1.1 · Load the three matched matrices

Load and check shapes — RNA and protein are gene × sample matrices sharing the same sample columns:

```
rna, prot, mut_freq, meta = load_cptac()
print(rna.shape, prot.shape, mut_freq.shape)  # genes x samples
```

Step 1.2 · Harmonize identifiers

Intersect on a common gene key and check how many genes survive — always the first reality check.

```
common = rna.index.intersection(prot.index).intersection(mut_freq.index)
print(f"surviving genes: {len(common)}")
```

Step 1.3 · Sample-level RNA–protein correlation

With matched samples, ask per gene: does protein track RNA across patients? Compute Spearman correlation for each gene.

```
corr = pd.Series({g: stats.spearmanr(rna.loc[g, samples],
                                     prot.loc[g, samples]).statistic
                  for g in common})
print(corr.median(), corr.min(), corr.max())
```

What you should see — and why it matters

The median correlation is well below 1.0. Most genes' protein levels only partly follow their RNA — the empirical reason you cannot treat RNA as a stand-in for protein. A gene where protein doesn't track RNA is a biology signal (translational buffering, turnover), not bad data.

Step 1.4 · Per-layer signal, then the multi-evidence score

Each layer nominates candidates via its effect size; the genomic layer uses mutation frequency. Normalize each layer to a common 0–1 scale with `rank_percentile` and take the equal-weighted sum:

```
scored["score"] = multi_evidence_score(
    scored, ["transcriptomic", "proteomic", "genomic"], EQUAL_WEIGHTS)
ranked_cptac = scored.sort_values("score", ascending=False)
```

Continuity check: TP53 / p53 — the `P04637` / PDB `1TUP` protein from Module 1 — lands near the top, exactly as you'd expect for breast cancer. Recognizing it is a good sanity check that the pipeline is behaving.

Part 2 — Find and load real CPTAC data (together)

Now we swap the synthetic data for the real thing. We'll do this together in class — **not graded** — so you leave knowing how to get real multi-omics data yourself. CPTAC breast data is open access (no data-use agreement).

Step 2.1 · Download from LinkedOmics

- Go to **LinkedOmics** (`linkedomics.org`) and select the **CPTAC Breast Cancer (BRCA)** cohort.
- Download the gene-level **RNAseq** matrix and the gene-level **Proteome** matrix (both gene × sample).

Step 2.2 · Format into the three expected files

Save them tab-separated with genes as the row index and the exact names the notebook expects. If a download has samples in rows, transpose it first:

```
import pandas as pd
rna = pd.read_csv("linkedomics_brca_rnaseq.tsv", sep="\t", index_col=0)
# genes must be the row index; if samples are rows instead, transpose:
# rna = rna.T
rna.to_csv("data/cptac_brca_rna.tsv", sep="\t")
# repeat for the proteome -> data/cptac_brca_protein.tsv
# mutation: one row per gene with its mutation frequency (or CNV magnitude)
#           -> data/cptac_brca_mutation.tsv
```

Step 2.3 · Re-run Part 1 on the real numbers

With the real files in `data/`, do `Kernel → Restart & Run All` and watch Part 1 again — the notebook now loads your files instead of the demo. Compare: is the RNA–protein correlation still modest? Does TP53 still surface near the top?

> **One honest wrinkle**
> The real download has no tumor-vs-normal labels, so the notebook ranks by overall abundance rather than a differential effect. That's fine for seeing the pipeline run on real data. If you want a true tumor-vs-normal signal, ask me — it needs matched normal samples.

Part 3 — Your assignment: Alzheimer's data (graded)

Now transfer the workflow to Alzheimer's on your own. The three matrices (from Canvas) are **gene-level summaries from different cohorts** — there are no shared samples, so you integrate at the gene level. Complete each `# TODO` cell in the notebook. Hints below; not full answers. **Continuity:** expect APOE (`rs7412`) among your top hits.

3.1 · Load and inspect

Call `load_ad()` and print the columns and shape of each table so you know your join key before touching anything.

3.2 · Harmonize on the gene symbol

Hint. Rename the `log2fc`/`pval` columns before merging so RNA and protein don't collide, then merge all three on gene. Report the surviving-gene count — if it's suspiciously low, your symbols aren't matching (see Troubleshooting).

```
tx = tx.rename(columns={"log2fc": "rna_lfc", "pval": "rna_p"})
pr = pr.rename(columns={"log2fc": "prot_lfc", "pval": "prot_p"})
df = ...   # merge tx, pr, gw on "gene"
```

3.3 · Sign-agreement concordance

Hint. You can't correlate across samples here, so concordance becomes direction agreement. Compare `np.sign(df["rna_lfc"])` with `np.sign(df["prot_lfc"])` to make a boolean `concordant` column. Genes strong at RNA but flat/opposite at protein are the interesting discordant ones — you'll write about one.

3.4 · Multi-evidence score

Hint. Build a magnitude per layer (`abs` of the RNA and protein effects; `neglog10p` for genomics), then call `multi_evidence_score(...)` with `EQUAL_WEIGHTS`. You may change the weights — but if you do, justify it in your report.

3.5 · Rank and export

Hint. Sort by score, take the top ~15, and export the ranked table to `targets_ad.csv`. **This CSV is the literal hand-off to Week 3**, so it has to be clean. Check whether known AD genes (APOE, TREM2, BIN1, CLU, PICALM…) are recovered, and look at any `concordant == False` genes in your top hits.

3.6 · Interpretation (this goes in your report — the 40%)

Answer these four in your 3–4 page report:

- **Weighting.** Argue for equal weights or for up-weighting a layer (e.g. GWAS as germline/causal-leaning vs. transcriptomics as possibly downstream). Reasoned vs. unreasoned is what's graded.
- **Top targets.** Which known AD genes did you recover? Any non-obvious hit worth a second look?
- **Read a discordant gene.** Pick a `concordant == False` gene in your top hits. What biology could explain RNA and protein disagreeing?
- **Limitation.** This was gene-level, cross-cohort integration, so you cannot make per-patient claims. Contrast with the matched CPTAC case from Parts 1–2.

Troubleshooting

Symptom	Likely cause & fix
FileNotFoundError from load_ad()	The three AD matrices aren't in data/. Check the folder name and that files sit next to the notebook, not in a subfolder of a subfolder.
Part 1 prints "SYNTHETIC demo data"	Expected if the Part 1 files aren't in data/ — the notebook falls back to its own demo and still runs.
Real CPTAC file looks sideways	LinkedOmics may put samples in rows. Transpose so genes are the row index: df = df.T before saving.
Merge gives 0 (or very few) rows	Gene symbols don't match across files — trailing spaces, case, or version suffixes. Try `df["gene"].str.strip().str.upper()` on each before merging.
spearmanr returns nan	A gene has no variance across samples (all identical values). Fine to leave as NaN; just don't let it break your sort.
KeyError on a column name	Your file's column header differs from what the notebook expects. Rename to `gene / log2fc / pval / neglog10p` exactly.
Notebook won't run top-to-bottom	You skipped a cell. Use `Kernel → Restart & Run All` and fix the first error before moving on.

Submit (Part 3 only)

- `Kernel → Restart & Run All` — confirm the whole notebook runs cleanly top to bottom.
- Commit the notebook, `targets_ad.csv`, and a short README (your name + anything that didn't work) to your `biot6900` repo.
- Push, confirm the files appear on github.com, then **post your repo link on Canvas.**

Due Thursday, Sept 24 at 6:00 pm (start of Week 3). Late policy per syllabus: −10% up to 1 day, −20% up to 2 days, −50% up to 3 days, no credit after.

How Part 3 is graded (100 pts · course 60 / 40 split)

Component	Pts	Type
Data loaded + identifiers harmonized (join works, count reported)	20	execution
Concordance + multi-evidence score computed correctly	25	execution
Ranked target table produced and exported to CSV	15	execution
Weighting justified; top targets interpreted biologically	25	interpretation
Discordance read + gene-level limitation discussed; report readable	15	communication
Total	**100**	

Partial credit is awarded for a correct approach even with minor technical errors — if a step wouldn't run, say what you were trying to do and what happened. Explaining your reasoning earns credit.